

**Jurnal Praktikum
Sistem Operasi
Modul 13: Perintah Dasar Linux**

**Nama : M Faris
NIM : 2311104017**

I. Perintah-Perintah Dasar Linux (Total 83 poin)

Untuk setiap soal, lakukan screenshot pada perintah yang dilakukan dan hasil yang ada pada terminal.

1. Terminal (4 poin)

Pada terminal Linux terdapat *command prompt* seperti ini:

```
ubuntu@localhost:~$
```

Catatan: ini hanyalah contoh, setiap komputer akan memiliki *command prompt* yang berbeda-beda.

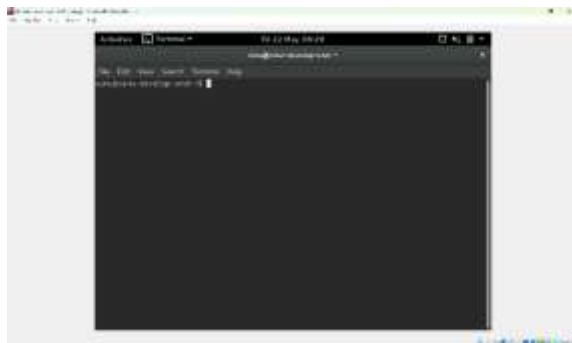
Pada linux command prompt biasanya dalam bentuk

```
username@host : working_directory $
```

Maka, arti dari `ubuntu@:~$` adalah username “ubuntu” yang berada komputer bernama “localhost”, ada pada direktori ~. Tanda \$ menunjukkan bahwa “ubuntu” adalah user biasa. Jika root yang mengeksekusi maka \$ akan berubah menjadi #.

Pertanyaan:

- a. Jalankan dan screenshot terminal Anda!



- b. Jelaskan arti dari *command prompt* milik Anda!

Arti command prompt `xinu@xinu-develop-end:~$` adalah: xinu merupakan username, xinu-develop-end adalah hostname (nama komputer), ~ menunjukkan posisi saat ini ada di dalam direktori *home*, dan \$ menandakan bahwa *user* login sebagai pengguna biasa (bukan root)

2. Perintah pertama (7 poin)

Setiap perintah pada Linux mempunyai bentuk umum:

```
nama_perintah option command_line_argument
```

maka perintah tersebut berarti:

```
nama_perintah = ls
```

option (atau switch) = -al

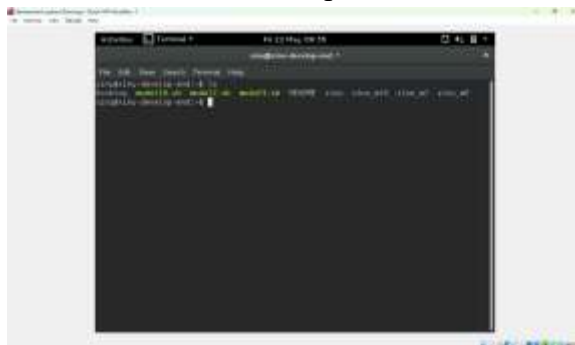
command_line_argument (atau parameter) = /

Catatan: beberapa perintah dapat berjalan tanpa adanya option maupun parameter, beberapa perintah hanya dapat berjalan jika diberikan parameter, perintah yang lain membutuhkan option dan parameter untuk dapat berjalan.

Perintah bersifat case sensitif: perintah ls berbeda dengan perintah LS

Pertanyaan:

- a. Jalankan dan screenshot perintah berikut ini: ls



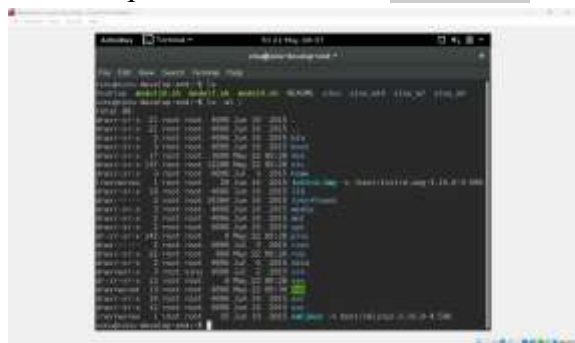
- b. Apakah option dan parameter dari perintah di atas?

Option = tidak ada, Parameter = tidak ada

- c. Apa fungsi dari perintah tersebut?

Fungsi perintah tersebut adalah menampilkan daftar file dan direktori di dalam lokasi aktif saat ini

- d. Jalankan perintah berikut ini: `ls -al /`



- e. Apakah option dan parameter dari perintah di atas?

Option = -al, Parameter = /

- f. Apa fungsi dari perintah tersebut?

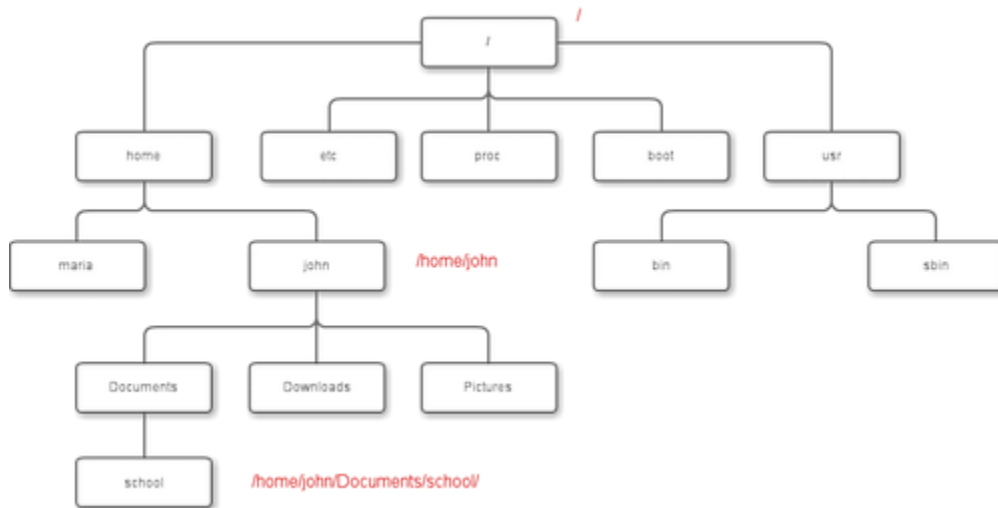
Fungsi perintah tersebut adalah menampilkan seluruh isi dari direktori root (/) secara mendetail, termasuk file yang tersembunyi (hidden files)

- g. Jelaskan mengapa perintah pada a dan e mempunyai hasil yang berbeda!

Perintah (a) dan (e) berbeda hasilnya karena perintah (a) hanya memanggil isi folder saat ini tanpa detail, sedangkan (e) memanggil isi dari folder root secara spesifik beserta detail hak akses dan ukuran file berkat adanya argumen parameter dan option

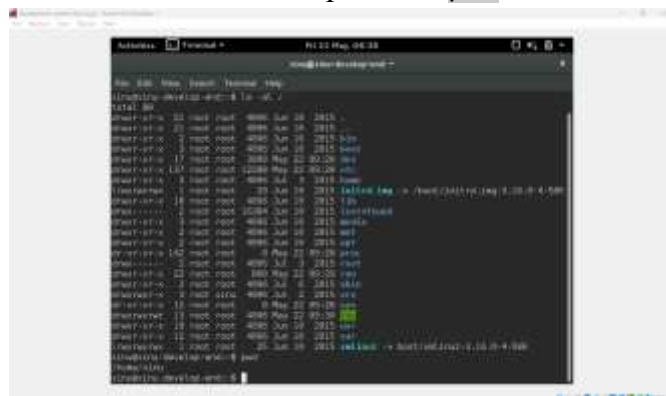
3. Pohon file (6 poin)

Melihat struktur direktori dan file pada Linux sama seperti melihat file dan direktori (folder) yang berada pada GUI. Pada Linux terdapat direktori utama yaitu “/” (root). Di bawah direktori root terdapat berbagai direktori yang lain seperti: home, etc, usr, dll. Contoh: program firefox terdapat pada path /usr/lib/firefox, berarti untuk mengakses firefox kita berjalan dari root kemudian direktori “usr” dilanjutkan direktori “lib” baru dapat mengeksekusi firefox



Pertanyaan:

- Jalankan dan screenshot perintah: `pwd`



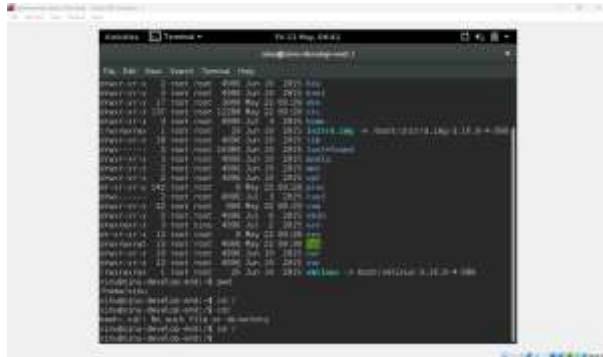
- Apakah option dan parameter dari perintah tersebut?
Option = tidak ada, Parameter = tidak ada

- c. Apa fungsi perintah tersebut?

Fungsi perintah ini adalah untuk mencetak absolute path atau lokasi lengkap direktori tempat user berada saat ini (Print Working Directory)

4. Perpindahan (6 poin)

- a. Jalankan dan screenshot perintah: `cd /`



- b. Apakah option dan parameter dari perintah tersebut?

Option = tidak ada, Parameter = /

- c. Apa yang dilakukan perintah tersebut?

Perintah tersebut memindahkan posisi direktori kerja saat ini ke direktori paling atas, yaitu root

5. Direktori khusus (6 poin)

`~` disebut **direktori home**. Misalkan user bernama linus, mempunyai home directory `/home/linus`, dengan melakukan `cd ~` maka sama dengan `cd /home/linus`.

`..` merepresentasikan **parent directory**. Jika kita berada dalam folder `/home` dan kemudian mengeksekusi perintah `cd ..` berarti kita berpindah dari direktori `/home` ke parentnya yaitu direktori `/`

`.` merupakan **current directory**, pada gambar pohon di atas jika kita berada pada folder `/home/john` maka untuk mengakses `school` dapat digunakan perintah `./Documents/school`

- a. Lakukan dan screenshot perintah `cd /` kemudian lakukan perintah `cd ~`. Jelaskan hasil dari keduanya!

```
xinu@xinu-develop-end:/$ cd /
xinu@xinu-develop-end:/$ cd ~
xinu@xinu-develop-end:~$
```

Perintah `cd /` memindahkan *user* ke root direktori, lalu perintah `cd ~` memindahkan *user* kembali ke direktori *home* miliknya (yaitu di `/home/xinu`)

- b. Lakukan perintah `cd /proc/self`. Buatlah perintah menggunakan `cd ..` agar dapat berpindah ke direktori `/` (root). Berapa kali perintah `cd ..` harus dieksekusi? Screenshot hasilnya!

```
xinu@xinu-develop-end:~$ cd /proc/self
xinu@xinu-develop-end:/proc/self$ cd ../../
xinu@xinu-develop-end:/$
```

Perintah `cd ..` harus dieksekusi sebanyak **1 kali** untuk bisa kembali dari `/proc/self` ke direktori `/` (root), atau bisa dipersingkat dengan satu perintah `cd ../../`.

6. Copy, rename dan delete file (screenshot setiap tahapan!) (14 poin)

Hint: gunakan **cp**, **mv**, dan **rm**

- Copylah file dari `/proc/cpuinfo` ke folder home Anda (`/home/user/`) menggunakan command pada terminal. Ganti user dengan username anda.
- Tunjukkan menggunakan perintah bahwa file tersebut benar-benar telah dicopy ke folder home Anda.

```
xinu@xinu-develop-end:/$ cp /proc/cpuinfo ~/
xinu@xinu-develop-end:/$ ls -l ~/cpuinfo
-r--r--r-- 1 xinu xinu 635 May 22 09:46 /home/xinu/cpuinfo
xinu@xinu-develop-end:/$
```

- Copy file dari `/proc/uptime` ke folder home Anda.
- Tunjukkan menggunakan perintah bahwa file tersebut benar-benar telah dicopy ke folder home Anda.

```
xinu@xinu-develop-end:/$ ls -l ~/uptime
-r--r--r-- 1 xinu xinu 16 May 22 09:48 /home/xinu/uptime
xinu@xinu-develop-end:/$
```

- Hapuslah file `uptime` di folder home Anda.
- Tunjukkan menggunakan perintah bahwa file tersebut benar-benar telah dihapus ke folder home Anda.

```
xinu@xinu-develop-end:/$ rm ~/uptime
rm: remove write-protected regular file '/home/xinu/uptime'?
xinu@xinu-develop-end:/$ ls -l ~/uptime
-r--r--r-- 1 xinu xinu 16 May 22 09:48 /home/xinu/uptime
xinu@xinu-develop-end:/$
```

- Rename file `cpuinfo` di folder home Anda menjadi `infocpu`

```
xinu@xinu-develop-end:/$ mv ~/cpuinfo ~/infocpu
xinu@xinu-develop-end:/$
```

7. Membuat folder baru. (4 poin)

Hint: gunakan **mkdir**

Buatlah perintah-perintah sehingga dapat melakukan hal-hal berikut ini: (lakukan secara berurutan mulai dengan melakukan perintah **cd ~**)

- Buatlah folder baru dengan nama “nim_anda”.
- Buatlah di dalam folder “nim_anda”, folder baru dengan nama “nama_anda”.

```
xinu@xinu-develop-end:~$ mkdir 2311104017
xinu@xinu-develop-end:~$ mkdir 2311104017/"M Faris"
bash: mkdir: command not found
xinu@xinu-develop-end:~$ mkdir 2311104017/"M Faris"
xinu@xinu-develop-end:~$
```

8. Membaca manual (10 poin)

Hint: gunakan **man**

- Bukalah fungsi manual untuk perintah “**ls**”



```
ls(1)                                User Comments                                ls(1)
NAME
  ls - list directory contents

SYNOPSIS
  ls [OPTION]... [FILE]...

DESCRIPTION
  List information about the files (the current directory by default). Sort
  entries alphabetically if none of -effort, -sort, or --sort is specified.
  Mandatory arguments to long options are mandatory for short options too.

  -d, --dir
      do not ignore entries starting with .

  -l, --long
      do not list implicit . and ..

  --author
      with -l, print the author of each file

  -R, --recursive
      print C-style escapes for nonprintable characters

  --block-size[=BLOCK_SIZE]
      print sizes in BLOCK_SIZE blocks
```

- Apa fungsi perintah “**ls**”?
Fungsi ls: Menampilkan daftar isi direktori.
- Siapakah pencipta perintah “**ls**”?
Pencipta ls: Richard Stallman dan David MacKenzie
- Apakah arti dari **-h** dari manual **ls**?
Arti -h: *Human-readable*, menampilkan ukuran file dalam format yang mudah dibaca (misal: 1K, 2M)
- Option apa yang harus digunakan agar dapat melihat direktori secara rekursif?
Option rekursif: -R atau --recursive
- Bukalah fungsi manual untuk perintah “**cp**”



```
cp(1)                                User Comments                                cp(1)
NAME
  cp - copy files and directories

SYNOPSIS
  cp [OPTION]... [-t TARGET]... FILE...

DESCRIPTION
  Copy SOURCE to DEST, or multiple SOURCE(s) to DIRECTORY.
  Mandatory arguments to long options are mandatory for short options too.

  -d, --directories-only
      don't copy the file data, just the attributes

  --attributes-only
      don't copy the file data, just the attributes

  --backup[=CONTROL]
      make a backup of each existing destination file

  -k      like --backup but does not remove an argument

  --help
      display this help message and exit
```

- Apa fungsi perintah “**cp**”?
Fungsi cp: Menyalin file dan direktori.

- h. Siapakah pencipta perintah “cp”?
- Pencipta cp: Torbjorn Granlund, David MacKenzie, dan Jim Meyering.
- i. Apakah arti -v dalam perintah “cp”?
- Arti -v: *Verbose*, menjelaskan detail proses apa yang sedang dilakukan
- j. Jika ingin interaktif, option apa yang harus digunakan?
- Option interaktif: -i atau -interactive
9. Pipe (12 poin)
- a. Lakukan perintah ini cat /etc/passwd dan screenshot hasil perintah tersebut!

```
xinu@xinu-develop-end:~$ cat /etc/passwd
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
avahi:x:105:112:Avahi sDNS daemon,,,:/var/run/avahi-daemon:/bin/false
avahi-autoipd:x:106:117:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/bin/false
colord:x:109:113:colord colour management daemon,,,:/var/lib/colord:/bin/false
pulse:x:112:122:PulseAudio daemon,,,:/var/run/pulse:/bin/false
usbmux:x:116:46:usbmux daemon,,,:/var/lib/usbmux:/bin/false
tftp:x:129:127:tftp daemon,,,:/srv/tftp:/bin/false
xinu@xinu-develop-end:~$
```

- b. Apa fungsi perintah cat?
- Fungsi cat adalah membaca isi suatu file teks dan mencetaknya seluruhnya ke layar
- c. Lakukan perintah cat /etc/passwd | grep daemon dan screenshot hasil perintah tersebut!

```
xinu@xinu-develop-end:~$ cat /etc/passwd | grep daemon
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
avahi:x:105:112:Avahi sDNS daemon,,,:/var/run/avahi-daemon:/bin/false
avahi-autoipd:x:106:117:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/bin/false
colord:x:109:113:colord colour management daemon,,,:/var/lib/colord:/bin/false
pulse:x:112:122:PulseAudio daemon,,,:/var/run/pulse:/bin/false
usbmux:x:116:46:usbmux daemon,,,:/var/lib/usbmux:/bin/false
tftp:x:129:127:tftp daemon,,,:/srv/tftp:/bin/false
xinu@xinu-develop-end:~$
```

- d. Lakukan perintah cat /etc/passwd | grep root dan screenshot hasil perintah tersebut!

```
xinu@xinu-develop-end:~$ cat /etc/passwd | grep root
root:x:0:0:root:/root:/bin/bash
xinu@xinu-develop-end:~$
```

- e. Lakukan perintah cat /etc/passwd | grep nobody dan screenshot hasil perintah tersebut!

```
xinu@xinu-develop-end:~$ cat /etc/passwd | grep nobody
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
xinu@xinu-develop-end:~$
```

- f. Apakah fungsi perintah “ | grep daemon”?
- Fungsi | grep daemon adalah menggunakan *pipe* | untuk meneruskan *output* dari cat, lalu memfilternya dengan grep agar terminal hanya menampilkan baris yang mengandung kata "daemon"

10. Redirection (14 poin)

- a. Lakukan perintah dan jelaskan hasilnya

```
cd /
```

```
ls -al > /home/user/result.txt
```

Ganti user dengan username ubuntu anda.

```
xinu@xinu-develop-end:~$ cd /
xinu@xinu-develop-end:/$ ls -al > ~/result.txt
xinu@xinu-develop-end:/$
```

- b. Dimana file result.txt berada?

Hasil perintah `ls -al` tidak ditampilkan di layar, melainkan diarahkan dan disimpan ke dalam file teks bernama result.txt. File tersebut berada di dalam direktori *home* milik user (/home/xinu/).

- c. Lakukan perintah dan jelaskan hasilnya

```
cd /etc
```

```
ls -al > /home/user/result.txt
```

Ganti user dengan username ubuntu anda.

- d. Apakah fungsi dari perintah `>`?

Hasil yang baru akan menimpa seluruh isi file result.txt yang lama secara total.

Fungsi `>` adalah untuk membelokkan *output* ke dalam sebuah file, dan jika file sudah ada, isinya akan di-*overwrite* (ditimpa).

- e. Lakukan perintah dan jelaskan hasilnya

```
cd /
```

```
ls -al >> /home/user/result1.txt
```

Ganti user dengan username ubuntu anda.

Hasil dari / dimasukkan dan membuat file baru result1.txt

- f. Lakukan perintah dan jelaskan hasilnya

```
cd /etc
```

```
ls -al >> /home/user/result1.txt
```

Ganti user dengan username ubuntu anda.

- g. Apakah perbedaan perintah `>` dan `>>`?

Hasil dari direktori /etc ditambahkan di baris paling bawah dari file result1.txt.

Perbedaan utamanya adalah: `>` menimpa (*overwrite*) seluruh isi file, sedangkan `>>` menambahkan teks di akhir file tanpa menghapus teks aslinya (*append*).

II. Kompile Source Code (Total 17 Poin)

Pastikan gcc sudah terinstall pada Ubuntu anda menggunakan `gcc --version`
Panggil atau hubungi asisten praktikum apabila gcc belum terinstall!

Hint:

`gcc -o output file`
`./`

Screenshot setiap tahapan yang anda lakukan terhadap Ubuntu!

1. Buatlah file dengan nama `2_1.c` yang berisi:

```
#include <stdio.h> //librari untuk printf()

int main(){ //main()
int a = 1; // variabel a diinisiasi dengan 1 <stdio
int b = 2; // variabel b diinisiasi dengan 2

printf ("Hello World!\n"); // tampilkan "Hello World" pada
layar

printf ("a = %d, b = %d, a + b = %d\n", a, b, a+b);
    // tampilkan a = 1, b = 2, a + b = 3
}
```

2. Kompile source code tersebut menggunakan gcc! Nama output program adalah `2_1` (bukan `a.out`). Tuliskan perintah untuk mengkompile source code tersebut!
3. Jalankan program yang baru saja Anda kompilasi. Tuliskan perintah untuk menjalankan program tersebut!
4. Buatlah file dengan nama `2_2.c` yang berisi:

```

#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <string.h>
#include <errno.h>

int main(int argc, char *argv[])
{
    int fd;

    if(2 != argc){
        printf("\n Penggunaan 'myopen nama_file' \n");
        return 1;
    }

    errno = 0;
    fd = open(argv[1],O_RDONLY);

    if(-1 == fd){
        printf("\n myopen gagal membuka file %s dengan pesan
error:
                [%s]\n", argv[1],strerror(errno));
        return 1;
    }else{
        printf("\n myopen berhasil membuka file %s\n",
argv[1]);
    }

    return 0;
}

```

5. Kompilasi source code tersebut menggunakan gcc! Nama output program adalah "myopen". Tulis perintah untuk mengkompilasi source code tersebut.
6. Jalankan program myopen yang baru saja Anda buat! Tuliskan perintah untuk menjalankan program myopen.
7. Jelaskan apa yang dilakukan program tersebut!